Dissipative Avoidance Feedback (DAF) — A Beginner-Friendly Explanation

> **Viewing:** Open in VS Code and press **Cmd+Shift+V** to open the preview with rendered equations.

The Big Picture

Imagine you have a robot (like a drone or a wheeled vehicle) that needs to travel from point A to point B in a room full of obstacles it has never seen before. The robot can only "see" nearby obstacles using sensors like LiDAR or a depth camera.

The classic approach to this problem is called **Artificial Potential Fields (APF)**: you create an invisible "force field" around each obstacle that *pushes* the robot away, like magnets repelling each other. The problem? This push is sudden and aggressive — the robot jerks around, wastes energy, and can get stuck.

This paper proposes a new idea: **Dissipative Avoidance Feedback (DAF)**. Instead of *pushing* the robot away from obstacles, DAF *slows it down* as it approaches one — like the air resistance you feel when you put your hand out of a car window. The closer you get, the more the braking kicks in, but only in the direction of the obstacle. If you're sliding *along* a wall (not into it), the braking doesn't activate. This leads to smoother, more natural motion.

Key differences from APF at a glance

Feature	APF (Classical)	DAF (This Paper)
Avoidance mechanism	Repulsive *force* (pushes away)	Dissipative *braking* (slows down)
Depends on	Position only	Position **and** velocity
Motion quality	Jerky, aggressive	Smooth, natural
Local minima	Prone to getting stuck	Much less likely
What it needs from sensors	Distance + direction to obstacle	Distance + direction to obstacle

The Paper's Logic — Before the Equations

Before diving into equations, here is the narrative thread:

1. **Define the world** (Eqs. 1-4) — formalise what "obstacle", "free space", and "safe" mean mathematically.
2. **Model the robot** (Eq. 5) — state that we control acceleration, not position.
3. **Design the energy function** (Eqs. 6-8) — borrow a framework from physics (Rayleigh dissipation) to build the braking idea.
4. **Derive the controller** (Eqs. 9-13) — let physics mechanics produce the control law automatically.
5. **Find the weak spots** (Eqs. 14-15) — show where the robot *could* get stuck.
6. **Compare to APF** (Eqs. 16-18) — benchmark against the standard method.
7. **Prove safety** (Eqs. 19-27) — rigorously verify the robot never collides.

Master Symbol Table

Every symbol in the paper, grouped by category.

Spaces and Sets

Symbol	Type	Meaning
$\mathbb{R}$	set	Real numbers
$\mathbb{R}_{>0}$	set	Positive real numbers (strictly > 0)
$\mathbb{N}$	set	Natural numbers {1, 2, 3, ...}
$\mathbb{R}^n$	set	n-dimensional Euclidean space — the world the robot lives in
n	$\mathbb{N}$	Dimension of the workspace (2 for planar, 3 for spatial)
$\mathcal{W}$	$\subset \mathbb{R}^n$	Workspace — the closed, bounded region the robot operates in (the "room")
$\mathcal{O}_i$	$\subset \mathcal{W}$	The i-th obstacle — a compact (closed + bounded) subset
m	$\mathbb{N}$	Number of obstacles
$\mathcal{X}$	$\subset \mathcal{W}$	Free space — where the robot is allowed to be
$\mathcal{X}^{\mathcal{C}}$	$\subset \mathbb{R}^n$	Complement of free space = the obstacle region
$\mathcal{X}_\epsilon$	$\subset \mathcal{X}$	Practical free space — free space shrunk by $R + \epsilon$
$\partial\mathcal{X}_\epsilon$	set	Boundary of practical free space

Symbol	Type	Meaning
$\mathrm{int}(\cdot)$	operator	Topological interior of a set
$\partial(\cdot)$	operator	Boundary of a set
$\overline{\mathcal{A}}$	set	Closure of $\mathcal{A}$ — the set including its boundary
$\mathcal{A}^{\mathcal{C}}$	set	Complement of $\mathcal{A}$ — everything NOT in $\mathcal{A}$
$B(p, r)$	set	Open Euclidean ball: all points within distance r of p
$\mathcal{E}$	set	Set of undesired equilibrium points (Eq. 14)

Robot State Variables

Symbol	Type	Meaning
p	$\in \mathbb{R}^n$	**Position** of the robot's centre
v	$\in \mathbb{R}^n$	**Velocity** of the robot ($\dot{p} = v$)
u	$\in \mathbb{R}^n$	**Control input** — the acceleration we command ($\dot{v} = u$)
$\dot{p}$	$\in \mathbb{R}^n$	Time derivative of position = velocity
$\dot{v}$	$\in \mathbb{R}^n$	Time derivative of velocity = acceleration
p_d	$\in \mathbb{R}^n$	**Desired target position** (the goal)
R	$\in \mathbb{R}_{>0}$	**Radius** of the ball-shaped robot

Distance and Geometry Variables

Symbol	Type	Meaning
$d^0_{\mathcal{A}}(p)$	$\mathbb{R}_{\geq 0}$	Unsigned distance from point p to the closest point in set $\mathcal{A}$: $\inf_{y \in \mathcal{A}} \lvert y - p \rvert$
$d_{\mathcal{A}}(p)$	$\mathbb{R}$	**Oriented (signed) distance** function: $d^0_{\mathcal{A}}(p) - d^0_{\mathcal{A}^C}(p)$. Positive outside $\mathcal{A}$, negative inside.
$d_{\mathcal{X}^C}(p)$	$\mathbb{R}$	Oriented distance from p to the obstacle region. Positive when p is in free space.

Symbol	Type	Meaning
$d(p)$	$\mathbb{R}$	**Shorthand**: $d_{\mathcal{X}^C}(p) - (R + \epsilon)$. This is 0 at the boundary of $\mathcal{X}_\epsilon$ and positive inside $\mathcal{X}_\epsilon$. **This is the distance the controller actually uses.**
$\dot{d}(p, v)$	$\mathbb{R}$	Time derivative of $d(p) = \eta(p)^\top v$. Rate of change of distance to obstacle. Negative means approaching.
$\ddot{d}$	$\mathbb{R}$	Second time derivative of $d(p)$. Acceleration of the distance.
$\mathbf{P}_{\mathcal{A}}(p)$	$\in \mathbb{R}^n$	**Projection** of p onto set $\mathcal{A}$ — the closest point in $\mathcal{A}$ to p
$\mathbf{P}_{\partial\mathcal{A}}(p)$	$\in \mathbb{R}^n$	Projection onto the boundary of $\mathcal{A}$ — the nearest boundary point
$\mathbf{P}_{\partial\mathcal{X}}(p)$	$\in \mathbb{R}^n$	Nearest point on the obstacle boundary to the robot
$\eta(p)$	$\in \mathbb{R}^n$, $\lvert\eta\rvert = 1$	**Unit normal vector** $= \nabla d_{\mathcal{X}^C}(p)$. Points away from nearest obstacle toward robot. The robot's "danger compass."
$\mathbf{H}(p)$	$\mathbb{R}^{n\times n}$	**Hessian matrix** $= \nabla^2 d_{\mathcal{X}^C}(p)$. Encodes how the obstacle surface curves at the nearest point.
$\lambda_{\mathbf{H}}(p^*)$	$\mathbb{R}$	An eigenvalue of $\mathbf{H}(p^*)$. The non-zero eigenvalues equal the **principal curvatures** of the obstacle boundary at p^*.

Design Parameters (You Choose These)

Symbol	Type	Meaning	Typical Value (from paper)
ϵ	$\in \mathbb{R}_{>0}$	Safety margin beyond robot radius	0.06
ϵ_1	$\in \mathbb{R}_{>0}$	Inner braking zone boundary (below this, $\gamma = 1/d$)	0.3
ϵ_2	$\in \mathbb{R}_{>0}$	Outer braking zone boundary (above this, $\gamma = 0$)	0.6
k_1	$\in \mathbb{R}_{>0}$	**Position gain** — strength of goal-seeking spring	3
k_2	$\in \mathbb{R}_{>0}$	**Damping gain** — strength of velocity friction	2

Symbol	Type	Meaning	Typical Value (from paper)
k_3	$\in \mathbb{R}_{>0}$	**Avoidance gain** — strength of obstacle braking	8

Geometric Constants (Properties of the Workspace)

Symbol	Type	Meaning
h	$\in \mathbb{R}_{>0}$	**Positive reach** of $\mathcal{X}$ (Assumption 1): the largest h such that every point within distance h of $\mathcal{X}^C$ has a unique projection onto $\partial\mathcal{X}$. Roughly: "how close can you get before the nearest-obstacle direction becomes ambiguous (e.g., equidistant from two obstacles)."
ρ	$\in \mathbb{R}_{>0}$	**Smoothness radius** (Assumption 2): the distance within which $d_{\mathcal{X}^C}$, $\nabla d_{\mathcal{X}^C}$, and $\nabla^2 d_{\mathcal{X}^C}$ are all continuous. Roughly: "how close can you get before the distance function stops being smooth."

Energy Functions

Symbol	Type	Meaning
$L(p, v)$	$\mathbb{R}$	**Lagrangian** $= T(v) - U(p)$
$T(v)$	$\mathbb{R}_{\geq 0}$	Kinetic energy $= v^\top v/2 = \lvert v\rvert^2/2$
$U(p)$	$\mathbb{R}_{\geq 0}$	Potential energy $= (k_1/2)\lvert p - p_d\rvert^2$ — a quadratic bowl centred on the goal
$D(p, v)$	$\mathbb{R}_{\geq 0}$	**Total Rayleigh dissipation** — energy lost to friction per unit time
$D_s(v)$	$\mathbb{R}_{\geq 0}$	Stabilisation dissipation — always-on velocity damping
$D_a(p, v)$	$\mathbb{R}_{\geq 0}$	Avoidance dissipation — obstacle-aware directional braking
$\mathcal{L}(p, v)$	$\mathbb{R}_{\geq 0}$	**Lyapunov function** (Eq. 19) $= (k_1/2)\lvert p - p_d\rvert^2 + (1/2)\lvert v\rvert^2$ — total energy used for stability proof

Proof-Specific Variables

Symbol	Type	Meaning
$\Phi(d, \dot{d})$	$\mathbb{R}$	**Barrier term** $= \gamma(d) \cdot \dot{d}$. Combines proximity scaling with approach rate. Blows up as $d \to 0$, creating the collision barrier.
$\alpha(p, v)$	$\mathbb{R}$	**Bounded remainder** $= k_1 \eta(p)^\top (p - p_d) - v^\top \mathbf{H}(p) v$. Captures goal-pull in obstacle direction + curvature correction. Proved bounded — can't overwhelm the barrier.
α_Φ	$\mathbb{R}$	Limiting constant value of Φ at a stuck point. Equals $-\mu k_1 / k_3$.
μ	$\mathbb{R}_{>0}$	$\lvert p^* - p_d \rvert$ — distance from stuck point to goal
p^*	$\in \mathbb{R}^n$	An undesired equilibrium point in $\mathcal{E}$
J	$\mathbb{R}^{2n \times 2n}$	Jacobian matrix of the frozen system at $(p^*, 0)$
$\mathbf{S}$	$\mathbb{R}^{n \times n}$	Stiffness matrix $= k_1 \mathbf{I} + k_3 \alpha_\Phi \mathbf{H}(p^*)$
s_i	$\mathbb{R}$	i-th eigenvalue of $\mathbf{S} = k_1 + k_3 \alpha_\Phi \lambda^i_{\mathbf{H}}$
$\mathbf{I}$	$\mathbb{R}^{n \times n}$	Identity matrix
π_η	$\mathbb{R}^{n \times n}$	Projection onto the plane orthogonal to η: $\pi_\eta = I_n - \eta\eta^\top$

Every Equation Explained

Equation (1) — The unit normal vector

$$\nabla d_{\mathcal{A}^c}(p)$$

Why it's here: The very first thing the controller needs is a direction — *which way is the obstacle?* Without this, the robot can't know where to apply the brakes. This equation defines the mathematical object that answers that question.

What it is: A unit-length arrow pointing from the nearest obstacle surface toward the robot.

Intuition: If you're standing near a wall, this is the arrow pointing straight out of the wall toward you. It's the robot's "danger compass" — always pointing away from the nearest threat.

- If the robot is outside the obstacle, the arrow points *away* from the obstacle surface toward the robot.

- This vector is called $\eta(p)$ throughout the rest of the paper.

Variable drill-down:

Variable	What it is here
$\nabla d_{\mathcal{A}^C}(p)$	Gradient of the oriented distance to the complement of $\mathcal{A}$. This is a **unit vector** (length 1).
p	Any point in space
$\mathcal{A}$	Any closed set (in practice, $\mathcal{A} = \mathcal{X}^C$ = obstacle region)
$\mathcal{A}^C$	Complement of $\mathcal{A}$ (in practice, $\mathcal{A}^C = \mathcal{X}$ = free space)
$\mathbf{P}_{\partial\mathcal{A}}(p)$	The nearest point on the boundary of $\mathcal{A}$ to p
$p - \mathbf{P}_{\partial\mathcal{A}}(p)$	Vector from nearest boundary point to p
$\lvert p - \mathbf{P}_{\partial\mathcal{A}}(p)\rvert$	Length of that vector = distance to boundary
$\mathrm{int}(\mathcal{A}^C)$	Interior of complement = "strictly outside $\mathcal{A}$"
$\mathrm{int}(\mathcal{A})$	Interior of $\mathcal{A}$ = "strictly inside $\mathcal{A}$"

Key insight: When $\mathcal{A} = \mathcal{X}^C$ (obstacle region) and p is in free space ($\mathrm{int}(\mathcal{A}^C) = \mathrm{int}(\mathcal{X})$), this gives $\eta(p)$ — the unit vector pointing from the nearest obstacle surface toward the robot. When inside the obstacle (second case), it points *toward* the boundary (outward).

Equation (2) — Free space

$$\mathcal{X} := \mathcal{W} \setminus \bigcup_{i=1}^{m} \mathrm{int}(\mathcal{O}_i)$$

Why it's here: The paper needs a precise definition of where the robot is *allowed* to be before it can make any guarantees about keeping the robot there. Without defining "safe", you can't prove "safety."

What it is: The free space is the workspace minus all obstacle interiors.

Intuition: Take the room ($\mathcal{W}$), cut out all the obstacles — what's left ($\mathcal{X}$) is where the robot can go. The $\setminus$ symbol means "remove", and $\bigcup$ means "all together."

Variable drill-down:

Variable	What it is here
$\mathcal{X}$	Free space — the result
$\mathcal{W}$	Workspace boundary (the room walls)
$\setminus$	Set minus — "remove"
$\bigcup$	Union — "combine all"
$\text{int}(\mathcal{O}_i)$	Interior of obstacle i
m	Number of obstacles

Why $\text{int}(\mathcal{O}_i)$ and not just $\mathcal{O}_i$? The boundary of each obstacle is still part of the free space (the robot can touch the surface, it just can't go through it). In practice this distinction matters for the topology — $\mathcal{X}$ includes obstacle boundaries, making it a closed set.

Equation (3) — Practical free space

$$\mathcal{X}_\epsilon := \{p \in \mathbb{R}^n : d_{\mathcal{X}^c}(p) \geq R + \epsilon\}$$

Why it's here: Equation (2) treats the robot as a point. Real robots have physical size — a robot of radius R collides if its *centre* gets within R of an obstacle surface, even if the centre hasn't crossed the boundary. This equation shrinks the free space to account for that, plus adds a small extra safety margin ϵ.

What it is: The robot's *practical* safe zone — a version of free space where every obstacle is inflated by $R + \epsilon$.

Intuition: Draw a "keep out" ring around every obstacle slightly larger than the robot itself. The robot's centre must stay outside this ring. All safety proofs are made with respect to this shrunken space, not the raw geometry.

Variable drill-down:

Variable	What it is here
$\mathcal{X}_\epsilon$	Practical free space — the robot centre's actual safe zone
$p \in \mathbb{R}^n$	Any candidate position
$d_{\mathcal{X}^C}(p)$	Distance from p to the obstacle region $\mathcal{X}^C$. Large = far from all obstacles.
$R \in \mathbb{R}_{>0}$	Robot radius

Variable	What it is here
$\epsilon \in \mathbb{R}_{>0}$	Extra safety margin (design parameter)
$R + \epsilon$	Total inflation distance. Every obstacle effectively grows by this much.
$\subset \mathcal{X}$	$\mathcal{X}_\epsilon$ is a subset of $\mathcal{X}$ (strictly smaller than free space)

Geometric picture: Take every obstacle, inflate it by $R + \epsilon$ in all directions (Minkowski sum with a ball). The remaining space is $\mathcal{X}_\epsilon$. The robot's centre must stay in $\mathcal{X}_\epsilon$.

Equation (4) — Feasibility condition on ϵ

$$0 < \epsilon < \min(h, \rho) - R$$

Why it's here: The safety margin ϵ from Eq. (3) can't be arbitrarily large — if you inflate every obstacle too much, the inflated obstacles could overlap and block all paths. This equation sets an upper limit on ϵ to guarantee the practical free space is non-empty and navigable.

What it is: The valid range for choosing the safety buffer ϵ.

Intuition: h and ρ are geometric constants describing how "well-behaved" the obstacle shapes are (how close you can get before the math breaks down). The condition $\epsilon < \min(h, \rho) - R$ ensures there is always a valid path. If you violate this, the math has no guarantee of finding one.

Variable drill-down:

Variable	What it is here
ϵ	Safety margin being constrained
h	Positive reach — max distance where projection $\mathbf{P}_{\partial\mathcal{X}}$ is unique. If ϵ exceeds this, there can be points with two equidistant obstacles and η becomes undefined.
ρ	Smoothness radius — max distance where d, ∇d, $\nabla^2 d$ are all continuous. If ϵ exceeds this, the controller's derivatives blow up.
R	Robot radius
$\min(h, \rho)$	The tighter of the two constraints

Why both h and ρ? h ensures $\eta(p)$ exists and is unique (geometric requirement). ρ ensures $\eta(p)$ is smooth (analytic requirement). The controller needs both.

Equation (5) — Robot dynamics

$$\dot{p} = v, \quad \dot{v} = u$$

Why it's here: Now that we've defined the world, we need to say what kind of robot we're controlling. The paper makes a key choice here: control *acceleration* (second-order dynamics), not velocity directly. This matters because it's more realistic and it's what makes the Rayleigh dissipation framework applicable in the next step.

What it is: Newton's second law for a unit-mass robot.

Intuition:

- $\dot{p} = v$: position changes at the rate of velocity. If you move at 5 m/s, your position shifts 5 m every second.
- $\dot{v} = u$: velocity changes at the rate of acceleration. The control input u is the acceleration we choose.

This is called **second-order dynamics** because we control the second derivative of position. Most simpler work controls velocity directly (first-order), but that's less physically accurate and harder to apply the friction analogy to.

Variable drill-down:

Variable	Dimension	Meaning
$p \in \mathbb{R}^n$	n	Position of robot centre
$v \in \mathbb{R}^n$	n	Velocity of the robot
$u \in \mathbb{R}^n$	n	Control acceleration — what we **design**
$\dot{p}$	n	dp/dt — rate of change of position = velocity
$\dot{v}$	n	dv/dt — rate of change of velocity = acceleration

This is a **unit mass** assumption. For a robot of mass m, you'd have $\dot{v} = u/m$, but since m is constant you can absorb it into the gains.

Critical shorthand definitions (stated between Eq. 4 and Eq. 5)

These are defined inline in the text and used everywhere after:

Shorthand	Full form	Meaning
$d(p)$	$d_{\mathcal{X}^C}(p) - (R + \epsilon)$	**Effective distance** to the practical boundary. $d(p) = 0$ means the robot's inflated shell is touching the obstacle. $d(p) > 0$ means safe.

Shorthand	Full form	Meaning
$\eta(p)$	$\nabla d_{\mathcal{X}^C}(p)$	**Unit normal** pointing away from the nearest obstacle (the direction of "escape")
$\mathbf{H}(p)$	$\nabla^2 d_{\mathcal{X}^C}(p)$	**Hessian** of the distance function — an $n \times n$ matrix encoding the **curvature** of the nearest obstacle surface at the closest point. Its eigenvalues are $\{0, \kappa_1, \kappa_2, \ldots, \kappa_{n-1}\}$ where κ_i are the principal curvatures. $\mathbf{H}(p)\eta(p) = 0$ always (zero eigenvalue along the normal direction).

Equation (6) — Total Rayleigh dissipation function

$$D(p, v) := D_s(v) + D_a(p, v)$$

Why it's here: The paper wants to *design* the braking behaviour using a principled framework from physics, not just guess a formula. The Rayleigh dissipation function is that framework — it's a classical mechanics tool for describing energy loss due to friction. By choosing D carefully, the authors can derive the exact controller they want (Eq. 9-10) instead of writing it by hand.

What it is: The total energy dissipation (friction) in the system, split into two parts.

Intuition: Think of D as describing two types of drag:

- D_s — always-on drag, like air resistance. Slows the robot down everywhere.
- D_a — smart obstacle-aware drag. Only activates near obstacles when approaching them.

The separation is intentional: D_s handles stability (reaching the goal), D_a handles safety (not hitting obstacles).

Variable drill-down:

Variable	What it is here
$D(p, v)$	**Total dissipation function** — a scalar ≥ 0 quantifying total energy loss rate
$D_s(v)$	**Stabilisation dissipation** — velocity damping (friction), depends only on v
$D_a(p, v)$	**Avoidance dissipation** — obstacle-aware directional braking, depends on both p and v

The Rayleigh dissipation function is a classical mechanics concept: $\partial D / \partial v$ gives you the dissipative (friction) force.

Equation (7) — Stabilization dissipation

$$D_s(v) := \frac{k_2}{2} v^\top v$$

Why it's here: The robot needs something to guarantee it *stops* at the goal rather than oscillating forever. D_s is the simplest possible friction term that achieves this. Without it, the goal-seeking spring (term 1 in Eq. 10) would cause the robot to oscillate indefinitely back and forth.

What it is: Basic velocity damping — the standard friction term.

Intuition: $v^\top v = \|v\|^2$ is just the square of speed. So this is "drag proportional to speed squared." The gain k_2 controls how strong the damping is. Bigger k_2 = more friction = converges faster but reacts more sluggishly.

Variable drill-down:

Variable	What it is here
$k_2 > 0$	**Damping gain** — a positive scalar tuning how strong the velocity friction is
v	Velocity vector ($n \times 1$)
$v^\top$	Transpose of v ($1 \times n$ row vector)
$v^\top v$	Dot product $= \|v\|^2 = v_1^2 + v_2^2 + \ldots + v_n^2$ (scalar, always ≥ 0)
$(k_2/2)v^\top v$	Quadratic velocity dissipation — the factor $1/2$ is a convention so that $\partial D_s/\partial v = k_2 v$ comes out clean

When you take $\partial D_s/\partial v = k_2 v$, this produces the $-k_2 v$ damping term in the controller.

Equation (8) — Avoidance dissipation

$$D_a(p, v) := \frac{k_3}{2} \gamma(d(p))\, v^\top \eta(p)\, \eta(p)^\top v$$

Why it's here: This is the core novelty — the paper's main contribution expressed as an energy function. By encoding the desired obstacle-braking behaviour *here* (as a dissipation function) rather than writing the force directly, the authors can use Eq. (9) to derive the correct dynamics automatically, including cross-terms that would be easy to miss by hand.

What it is: The "smart brake" — directional, proximity-scaled friction near obstacles.

Intuition — piece by piece:

- $\eta(p)^\top v$ — dot product of velocity with the obstacle direction. This is the component of the robot's velocity aimed *toward* the obstacle. If moving parallel to a wall, this is zero. If moving head-on into a wall, this is large.
- $v^\top \eta(p)\,\eta(p)^\top v = (\eta^\top v)^2$ — squaring that component ensures the energy is non-negative (energy must always be positive or zero).
- $\gamma(d(p))$ — scales the braking by proximity. Far away: $\gamma = 0$, no effect. Very close: γ is large, strong braking.

Combined: D_a is large only when the robot is close to an obstacle AND moving toward it. It's zero otherwise. This is exactly the "smart brake that only engages on a collision course."

Variable drill-down:

Variable	Type	What it is here
$k_3 > 0$	Scalar gain	**Avoidance gain** — how aggressively the obstacle braking engages
$\gamma(\cdot)$	Function $\mathbb{R}_{>0} \to \mathbb{R}_{\geq 0}$	**Proximity scaling function** (defined in Eq. 11) — 0 when far, $\to \infty$ when close
$d(p)$	Scalar	Effective distance to nearest obstacle boundary (shorthand defined above)
$\gamma(d(p))$	Scalar ≥ 0	The proximity scaling evaluated at the current distance — "how close are we?"
$\eta(p)$	$n \times 1$ unit vector	Normal pointing away from nearest obstacle
$\eta(p)^\top$	$1 \times n$ row vector	Transpose of η
$\eta(p)^\top v$	**Scalar**	Dot product = component of velocity in the obstacle-normal direction. **This is the approach speed.** Positive = moving away, negative = moving toward.
$v^\top \eta(p)$	Same scalar	$= \eta(p)^\top v$ (dot product is commutative)
$v^\top \eta(p)\,\eta(p)^\top v$	Scalar ≥ 0	$= (\eta^\top v)^2$ — the **squared approach speed**. Always non-negative.
$\eta(p)\eta(p)^\top$	$n \times n$ matrix	**Projection matrix** onto the obstacle-normal direction. Rank-1 matrix. When you multiply it by v, you get the component of v in the η direction.

The key insight: $v^\top(\eta\eta^\top)v = (\eta^\top v)^2$ only captures the velocity component **toward/away from** the obstacle. Velocity parallel to the obstacle surface contributes **zero** to D_a. This is why DAF only brakes

when you're on a collision course.

Equation (9) — Euler-Lagrange equation

$$\frac{d}{dt}\frac{\partial L}{\partial v} - \frac{\partial L}{\partial p} + \frac{\partial D}{\partial v} = 0$$

Why it's here: This is the derivation step — the authors take the energy functions they designed (Eqs. 6-8) and feed them into this standard physics recipe to *automatically produce* the control law. It's why the paper uses the Rayleigh framework: you design the energy, then let mechanics do the algebra.

What it is: The Euler-Lagrange equation with Rayleigh dissipation — a recipe from classical mechanics.

Intuition: If you know a system's kinetic energy T, potential energy U, and friction D, plug them into this equation and it gives you the equations of motion. Here:

- $L = T - U = \frac{1}{2}\|v\|^2 - \frac{k_1}{2}\|p - p_d\|^2$ (kinetic minus potential)
- D = Eq. (6) above

Taking the derivatives and rearranging produces Eq. (10) directly.

Variable drill-down:

Variable	What it is here
$L(p, v)$	**Lagrangian** $= T(v) - U(p)$, where $T =$ kinetic energy, $U =$ potential energy
$T(v)$	Kinetic energy $= v^\top v/2 = \|v\|^2/2$ (unit mass)
$U(p)$	Potential energy $= k_1\|p - p_d\|^2/2$ — a quadratic bowl centred on the goal
$k_1 > 0$	**Position gain** — stiffness of the "spring" pulling toward the goal
$p_d \in \mathbb{R}^n$	**Desired position** (the goal/target)
$\partial L/\partial v$	Partial derivative of L w.r.t. $v = \partial T/\partial v = v$ (the "momentum" for unit mass)
$\frac{d}{dt}(\partial L/\partial v)$	Time derivative of momentum $= \dot{v}$ = acceleration
$\partial L/\partial p$	Partial derivative of L w.r.t. $p = -\partial U/\partial p = -k_1(p - p_d)$ (the restoring force)
$\partial D/\partial v$	Partial derivative of dissipation w.r.t. v = the dissipative force. From Eq. 7: $k_2 v$. From Eq. 8: $k_3\gamma(d(p))\eta(p)\eta(p)^\top v$.

Substituting and rearranging: $\dot{v} + k_1(p - p_d) + k_2 v + k_3\gamma\eta\eta^\top v = 0$, which gives Eq. (10).

Equation (10) — Closed-loop dynamics

$$\dot{v} = -k_1(p - p_d) - k_2 v - k_3 \gamma(d(p))\, \eta(p)\, \eta(p)^\top v$$

Why it's here: This is what comes out of Eq. (9) after substituting the energy functions. It's the complete description of how the robot accelerates at every instant. Each term has a clear physical origin from the design choices made in Eqs. (6)-(8).

What it is: The robot's acceleration as a function of its current state.

Intuition — three terms, three jobs:

Term	Formula	Job
Goal seeking	$-k_1(p - p_d)$	Spring pulling robot toward target. Stronger when further away.
Damping	$-k_2 v$	Friction slowing the robot. Prevents overshooting and oscillation.
Obstacle braking	$-k_3 \gamma \eta \eta^\top v$	Smart brake. Active only near obstacles when approaching.

Breaking down the third term step by step:

1. $\eta(p)^\top v \rightarrow$ scalar: how fast you're approaching the obstacle
2. $\eta(p) \times$ (that scalar) $\rightarrow$ vector: the braking force direction (along η)
3. $\gamma(d(p)) \times \rightarrow$ scale by proximity (stronger when closer)
4. $k_3 \times \rightarrow$ scale by the design gain
5. $- \rightarrow$ deceleration (opposing the approach velocity)

Equation (11) — The proximity function γ

$$\gamma(z) := \begin{cases} 0, & z \in [\epsilon_2, +\infty) \\ \phi(z), & z \in [\epsilon_1, \epsilon_2) \\ z^{-1}, & z \in (0, \epsilon_1) \end{cases}$$

Why it's here: The braking term in Eq. (10) needs a function that is zero far away and grows large near obstacles. This equation defines that function precisely. The paper also needs γ to be smooth (no sudden jumps) so the resulting forces don't cause jerky motion — that's why there's a blending zone in the middle.

What it is: The three-zone obstacle proximity scaling function.

Intuition:

Zone	Distance	γ	Effect
Far zone	$d \geq \varepsilon_2$	0	No braking at all.
Transition zone	$\varepsilon_1 \leq d < \varepsilon_2$	Smooth ramp $\phi(z)$	Braking gradually increases. No sudden jumps.
Close zone	$0 < d < \varepsilon_1$	$1/d \to \infty$	Braking grows without bound as $d \to 0$. This is the *mathematical barrier* — the robot can never actually reach $d = 0$ because the braking would need to be infinite.

The $1/d$ in the close zone is what guarantees safety in the proofs, but it also causes ODE stiffness in simulation — which is why the notebooks cap it at $1/\varepsilon_1$ in practice.

Variable drill-down:

Variable	What it is here
z	Input argument — will be substituted with $d(p)$ (effective distance to obstacle)
ϵ_2	**Outer activation threshold** — distance beyond which the obstacle braking is completely off
ϵ_1	**Inner activation threshold** — distance below which $\gamma = 1/z$ (the barrier zone)
$\phi(z)$	**Blending polynomial** — a smooth function connecting $1/\epsilon_1$ at $z = \epsilon_1$ to 0 at $z = \epsilon_2$. The paper uses a cubic polynomial (footnote 1) satisfying: $\phi(\epsilon_1) = 1/\epsilon_1$, $\phi'(\epsilon_1) = -1/\epsilon_1^2$, $\phi(\epsilon_2) = 0$, $\phi'(\epsilon_2) = 0$
$z^{-1} = 1/z$	The barrier function — blows up to $+\infty$ as $z \to 0^+$

Why three zones? The barrier $1/z$ guarantees safety mathematically (infinite braking prevents collision). But you can't switch it on abruptly — a discontinuity in γ would cause a discontinuity in u, which is physically impossible. The polynomial ϕ smoothly blends between "off" and "barrier mode."

Equation (12) — Conditions on ϵ_1 and ϵ_2

$$0 < \epsilon_1 < \epsilon_2 < \min(h, \rho) - (R + \epsilon)$$

Why it's here: Just as Eq. (4) constrained the safety margin ϵ, the paper now constrains the braking zone thresholds. The zone boundaries must be ordered ($\varepsilon_1 < \varepsilon_2$) and both must lie within the mathematically valid region of the free space.

What it is: The valid range for the braking zone parameters.

Intuition: You must have $\varepsilon_1 < \varepsilon_2$ for the transition zone to exist. Both must be small enough that the braking activates only within the region where the distance function (and its gradient η) is well-defined. If ε_2 is too large, γ would activate in regions where the math isn't valid.

Variable drill-down:

Variable	What it is here
ϵ_1, ϵ_2	Braking zone thresholds from Eq. 11
h	Positive reach (Assumption 1)
ρ	Smoothness radius (Assumption 2)
R	Robot radius
ϵ	Safety margin (Eq. 3)
$R + \epsilon$	Total inflation of obstacles
$\min(h, \rho) - (R + \epsilon)$	The remaining "valid" distance after accounting for robot size and safety buffer

---

Equation (13) — The DAF controller

$$u = -k_1(p - p_d) - k_2 v - k_3 \gamma(d(p))\, \eta(p)\, \eta(p)^\top v$$

Why it's here: After all the theory, the paper restates the controller explicitly as the *implementable recipe* — the thing an engineer actually codes up. It looks the same as Eq. (10), but here it's presented as: "given sensor readings, compute this and send it to the motors."

What it is: The control law — the acceleration command to apply at every timestep.

Intuition: To compute u at any moment the robot needs only:

1. Its own position p and velocity v — from onboard odometry/IMU.
2. Distance to the nearest obstacle $d(p)$ — from LiDAR/depth camera minimum range.
3. Direction to the nearest obstacle $\eta(p)$ — from the same sensor (direction of the nearest hit).

No map, no path planner, no knowledge of obstacle shape. Just three real-time sensor values.

Implementation mapping:

What to compute	How to get it	Sensor/source
p	Robot position	Odometry / GPS / motion capture
v	Robot velocity	Differentiate p or use IMU
p_d	Goal position	Given by the mission
$d(p)$	min_range $-(R+\epsilon)$	LiDAR minimum range reading
$\eta(p)$	Unit vector toward the minimum range reading	LiDAR bearing of closest hit
k_1, k_2, k_3	Positive scalar gains	Tuning parameters
$\gamma(d(p))$	Evaluate Eq. 11	Computed from $d(p)$
$\eta(p)^\top v$	Dot product (scalar)	Computed from η and v
$\eta(p)(\eta^\top v)$	Scale η by the dot product (vector)	Computed

---

Equation (14) — Undesired equilibrium points

$$\mathcal{E} := \{p \in \partial\mathcal{X}_\epsilon : (p - p_d) = \mu\,\eta(p),\ \mu > 0\}$$

Why it's here: Any stability proof must identify every point where the robot could stop moving — not just the goal, but also any "stuck" points. Eq. (14) characterises *all* the places where velocity could go to zero and the robot settles somewhere other than the goal.

What it is: The set of bad resting points on obstacle boundaries.

Intuition: A robot gets stuck on an obstacle boundary when the goal-seeking pull and the obstacle braking exactly cancel. Geometrically this happens when the robot, the obstacle surface normal, and the goal are all on the same line — with the obstacle between the robot and the goal. Imagine the goal is directly behind a wall; the robot slides along until it reaches the point directly in front of the goal, then the wall blocks it exactly.

The paper's goal is then to show that these $\mathcal{E}$ points are *unstable* — that a slight nudge will cause the robot to escape.

Variable drill-down:

Variable	What it is here
$\mathcal{E}$	The set of all undesired equilibrium positions
$\partial\mathcal{X}_\epsilon$	Boundary of practical free space — where $d(p) = 0$
$p - p_d$	Vector from the goal to the robot
$\mu > 0$	A positive scalar — the distance from goal to robot: $\mu = \lvert p - p_d \rvert$
$\eta(p)$	Unit normal pointing away from obstacle
$(p - p_d) = \mu\eta(p)$	The direction from goal to robot is exactly the obstacle normal — the goal, obstacle surface, and robot are **collinear**

Equation (15) — Curvature instability condition

$$\lambda_H(p^*) > \frac{1}{\|p^* - p_d\|}$$

Why it's here: Having identified the stuck points ($\mathcal{E}$), the paper now needs to show they're unstable so the robot doesn't stay there. Eq. (15) is the condition under which that instability is guaranteed. It connects obstacle shape (curvature) to goal distance.

What it is: The curvature threshold that makes stuck points unstable.

Intuition: λ_H measures how curved the obstacle surface is at the stuck point.

- **Curved obstacle (e.g. a ball):** The stuck point is like balancing on top of a basketball — any slight push and the robot slides off. The equilibrium is **unstable** (good).
- **Flat obstacle (e.g. a wall):** The stuck point is like sitting in a bowl — the robot stays. **Stable** (bad).

The right-hand side $1/\|p^* - p_d\|$ decreases as the goal moves farther away, so distant goals are easier to satisfy — meaning less curvature is needed.

Variable drill-down:

Variable	What it is here
p^*	An undesired equilibrium point ($p^* \in \mathcal{E}$)
$\lambda_{\mathbf{H}}(p^*)$	An eigenvalue of the Hessian matrix $\mathbf{H}(p^*) = \nabla^2 d_{\mathcal{X}^C}(p^*)$. The eigenvalues of $\mathbf{H}$ at a point on the boundary are the **principal curvatures** of the obstacle surface at the closest point.

Variable	What it is here
$\mathbf{H}(p^*)$	The $n \times n$ Hessian of the distance function at p^*. Since η is always a zero-eigenvector of $\mathbf{H}$ ($\mathbf{H} \cdot \eta = 0$), the non-zero eigenvalues are the curvatures in the tangent directions.
$\lvert p^* - p_d \rvert$	Distance from the stuck point to the goal
$1/\lvert p^* - p_d \rvert$	The curvature threshold — decreases as the goal gets farther away

Physical meaning: If the obstacle surface curves more sharply than the "curvature of the line of sight to the goal," the stuck point is unstable. For a sphere of radius r_{obs}, $\lambda_{\mathbf{H}} = 1/r_{\text{obs}}$, so the condition is $1/r_{\text{obs}} > 1/\|p^* - p_d\|$, i.e., $\|p^* - p_d\| > r_{\text{obs}}$ — which is always true since the goal must be outside the obstacle.

Equations (16)-(17) — APF potential energies (comparison only)

$$U_a(p) := \frac{k_a}{2}\|p - p_d\|^2$$

$$U_r(p) := \begin{cases} \frac{k_r}{2}\left(\frac{1}{d(p)} - \frac{1}{\epsilon_2}\right)^2, & d(p) \le \epsilon_2 \\ 0, & d(p) > \epsilon_2 \end{cases}$$

Why they're here: The paper benchmarks DAF against APF. To do that fairly, it needs to write out APF in the same mathematical language. These two equations are the APF "energy functions" — the equivalent of Eqs. (6)-(8) for APF. Comparing them reveals exactly *where* the designs differ.

What they are: Attractive and repulsive potential energies for the classical APF method.

Intuition:

- U_a is a bowl centred on the goal — the robot rolls downhill toward p_d. This part is identical in spirit to DAF's k_1 term.
- U_r is a hill around each obstacle — the robot is pushed uphill (away) as it approaches. This is fundamentally different from DAF: the pushing is always active based on position alone, not velocity. A robot moving *away* from an obstacle still feels the push. That's why APF causes jerky motion.

Variable drill-down:

Variable	What it is here
$U_a(p)$	APF **attractive** potential — quadratic bowl pulling toward goal

Variable	What it is here
$k_a > 0$	APF attraction gain (analogous to k_1 in DAF)
$U_r(p)$	APF **repulsive** potential — hill around obstacles pushing robot away
$k_r > 0$	APF repulsion gain
$1/d(p)$	Inverse distance — blows up near obstacles
$1/\epsilon_2$	Offset so the potential is zero at $d = \epsilon_2$
$(1/d - 1/\epsilon_2)^2$	Squared so $U_r \geq 0$ always

Key difference from DAF: U_r depends only on position. DAF's D_a depends on position AND velocity. This is why APF pushes even when the robot is moving away safely.

Equation (18) — APF controller (comparison only)

$$u_{\text{APF}} = -k_a(p - p_d) - k_v v + k_r F_r(p)$$

where the repulsive acceleration $F_r(p)$ is:

$$F_r := \begin{cases} \frac{k_r \eta(p)}{d^2(p)} \left(\frac{1}{d(p)} - \frac{1}{\epsilon_2} \right), & d(p) \leq \epsilon_2 \\ 0, & d(p) > \epsilon_2 \end{cases}$$

Why it's here: This is the APF control law written out, for direct comparison with Eq. (13). The structural similarity makes the difference obvious: the obstacle term F_r depends only on position, while DAF's obstacle term depends on both position and velocity.

What it is: The classical APF acceleration command.

Intuition: Same three terms as DAF — goal seeking, damping, obstacle. But the obstacle force $F_r(p) = \nabla U_r(p)$ is a *repulsive push that depends only on where you are*, not on how fast you're moving or in which direction. This is why APF pushes the robot sideways even when the robot is already moving away safely, causing unnecessary jerking.

Variable drill-down:

Variable	What it is here
u_{APF}	APF control acceleration
k_a	Attraction gain

Variable	What it is here
k_v	Velocity damping gain (analogous to k_2)
k_r	Repulsion gain
$F_r(p)$	Repulsive acceleration — derived as $-\nabla U_r(p)$
$\eta(p)/d^2(p)$	The repulsive force magnitude — direction η, intensity $1/d^2$
$(1/d - 1/\epsilon_2)$	Scale factor ensuring $F_r = 0$ at $d = \epsilon_2$

Structural comparison:

	DAF (Eq. 13)	APF (Eq. 18)
Goal term	$-k_1(p - p_d)$	$-k_a(p - p_d)$
Damping	$-k_2 v$	$-k_v v$
Obstacle term	$-k_3\gamma(d)\eta(\eta^\top v)$ — depends on v	$+k_r F_r(p)$ — depends only on p

---

Equation (19) — Lyapunov function

$$\mathcal{L}(p, v) = \frac{k_1}{2}\|p - p_d\|^2 + \frac{1}{2}\|v\|^2$$

Why it's here: All the proofs in Section IV need a mathematical "measuring stick" to show the robot is making progress toward safety and the goal. A Lyapunov function is that tool — if you can show it only decreases over time, you've proved convergence without solving the full dynamics. This specific choice is the total mechanical energy of the system (potential + kinetic), which is a natural fit for a damped spring system.

What it is: The total energy of the robot state — potential energy (distance to goal) plus kinetic energy (speed).

Intuition: Think of it as "how much work is left to do." High energy = far from goal and/or moving fast. Low energy = near goal and slowing down. The proof strategy: show $\dot{\mathcal{L}} \leq 0$ always, which means energy only flows out, the robot must eventually reach the goal.

Variable drill-down:

Variable	What it is here
$\mathcal{L}(p, v)$	**Lyapunov function** — total "energy" of the system
$(k_1/2)\|p - p_d\|^2$	Potential energy — distance to goal squared, scaled by k_1
$(1/2)\|v\|^2$	Kinetic energy — speed squared (unit mass)

Equation (20) — Time derivative of the Lyapunov function

$$\dot{\mathcal{L}}(p, v) = -k_2 \|v\|^2 - k_3 \gamma(d(p))\, \dot{d}^2(p, v)$$

Why it's here: This is the key step in the safety proof — computing $\dot{\mathcal{L}}$ and showing it's always ≤ 0. Both terms are non-positive (squares multiplied by positive gains), so energy is always decreasing. This single equation does most of the work in proving both stability (reaches goal) and safety (never collides).

What it is: The rate of energy change — always negative or zero.

Intuition:

- $-k_2\|v\|^2 \leq 0$: energy always decreases due to damping. Zero only when the robot is stationary.
- $-k_3\gamma\dot{d}^2 \leq 0$: additional energy drain when near obstacles and changing distance. $\dot{d} < 0$ means approaching (distance decreasing), so $\dot{d}^2$ captures the approach rate.

Since $\dot{\mathcal{L}} \leq 0$ everywhere, energy is bounded — the robot's position and velocity can never blow up. And since the obstacle barrier ($\gamma \to \infty$ as $d \to 0$) would require infinite energy to overcome, collision is impossible.

Variable drill-down:

Variable	What it is here
$\dot{\mathcal{L}}$	Time derivative of $\mathcal{L}$ — $d\mathcal{L}/dt$
$-k_2\|v\|^2$	Energy drain from velocity damping — always ≤ 0
$\dot{d}(p, v)$	**Time derivative of** $d(p)$ — rate of change of distance to obstacle. Computed as $\dot{d} = \eta(p)^\top v$ (the approach speed).
$\dot{d}^2$	Squared approach speed — always ≥ 0
$-k_3\gamma(d)\dot{d}^2$	Energy drain from obstacle braking — always ≤ 0

Equation (21) — Distance-to-obstacle dynamics

$$\ddot{d}(p, v) = -k_3 \Phi(d, \dot{d}) - k_2 \dot{d} - \alpha(p, v)$$

Why it's here: The Lyapunov proof shows the system is stable but doesn't directly say what happens to the distance $d(p)$. This equation tracks d explicitly over time so the paper can show $d(t) > 0$ for all time — i.e. the robot never reaches the obstacle surface. The Φ term is the key: it blows up as $d \to 0$, creating a mathematical barrier.

What it is: The acceleration of the robot's distance to the nearest obstacle.

Intuition: When d is small, Φ is huge (because $\gamma = 1/d$ is huge), creating an enormous deceleration that prevents d from decreasing further. It's like a car's brakes becoming infinitely powerful the closer you get to a wall — you can always stop in time.

Variable drill-down:

Variable	What it is here
$\ddot{d}$	Second time derivative of d — the "acceleration" of the distance-to-obstacle
$\Phi(d, \dot{d})$	**Barrier term** (defined in Eq. 22)
$k_2 \dot{d}$	Damping of the distance dynamics
$\alpha(p, v)$	**Bounded remainder** (defined in Eq. 23)

Equations (22)-(23) — Helper definitions

$$\Phi(d, \dot{d}) := \gamma(d(p))\, \dot{d}(p, v)$$

$$\alpha(p, v) := k_1 \eta(p)^\top (p - p_d) - v^\top H(p) v$$

Why they're here: Eq. (21) contains complex expressions that are easier to analyse when given names. These definitions factor out two reusable pieces for use in subsequent proofs.

What they are:

- Φ — the "barrier term": proximity scaling γ times the approach rate $\dot{d}$. This is the term that prevents collision.
- α — a bounded remainder: captures the goal-seeking component in the obstacle direction (first term) and a curvature correction (second term, involving the Hessian H of the distance function). The proof needs α to be bounded to ensure the barrier isn't overwhelmed.

Variable drill-down:

Variable	What it is here
$\Phi(d, \dot{d})$	**Barrier term** — product of proximity scaling and approach speed. When d is small and the robot approaches ($\dot{d} < 0$), Φ is large and negative, creating enormous positive $\ddot{d}$ (deceleration of approach).
$\gamma(d)$	Proximity function (Eq. 11)
$\dot{d}$	$\eta(p)^\top v$ — rate of change of distance (approach speed)
$\alpha(p, v)$	**Bounded remainder** — captures forces not related to the barrier
$k_1 \eta(p)^\top (p - p_d)$	Component of the goal-seeking force in the obstacle-normal direction. Positive when the goal is behind the obstacle (pulling toward it), negative when goal is on the same side.
$v^\top \mathbf{H}(p) v$	**Curvature correction** — a quadratic form in velocity involving the Hessian. Accounts for the fact that η changes direction as the robot moves along a curved surface. For a convex obstacle, this term is ≥ 0.
$\mathbf{H}(p)$	Hessian $\nabla^2 d_{\mathcal{X}^C}(p)$ — the $n \times n$ curvature matrix

---

Equation (24) — Characterisation of the stuck direction

$$(p - p_d) \to \mu\, \eta, \quad \text{with } \mu = \|p - p_d\| > 0$$

Why it's here: If the robot doesn't reach the goal (energy decreases but not to zero), where does it end up? This equation characterises that scenario: the vector from goal to robot aligns with the obstacle normal. It's the mathematical statement of "stuck behind an obstacle." The paper then uses this to verify the instability condition (Eq. 15).

What it is: The limiting alignment condition for stuck states.

Intuition: If you draw a line from the goal p_d through the robot's position p, it points in the same direction as $\eta(p)$ (the obstacle normal). In other words, the goal is directly behind the obstacle from the robot's perspective — the robot, the obstacle surface, and the goal are collinear.

Variable drill-down:

Variable	What it is here
$\to$	Converges to (as $t \to \infty$)
μ	The distance from goal to the stuck point: $\mu = \lvert p - p_d \rvert$

Variable	What it is here
η	The obstacle normal at the stuck point

Equation (25) — Frozen (asymptotic) system

$$\dot{p} = v, \quad \dot{v} = -k_1(p - p_d) - k_2 v - k_3 \alpha_\Phi \, \eta(p)$$

Why it's here: Proving instability of the stuck points requires analysing the system *near* those points. Near a stuck point, the time-varying barrier term Φ converges to a constant α_Φ — so the full nonlinear system can be approximated by this simpler, "frozen" linear system for the instability analysis.

What it is: A linearised approximation of the dynamics near a stuck equilibrium.

Intuition: Replace the complicated $\Phi(d(t), \dot{d}(t))$ with its limiting constant value α_Φ. Near a stuck point the system looks approximately like a spring-mass-damper with an extra constant force pushing toward the obstacle. Studying the eigenvalues of this simpler system tells us whether the stuck point is stable or unstable.

Variable drill-down:

Variable	What it is here
α_Φ	The **constant** that $\Phi(d, \dot{d})$ converges to at the stuck point. From the proof: $\alpha_\Phi = -\mu k_1 / k_3$, where $\mu = \lvert p^* - p_d \rvert$.
$k_3 \alpha_\Phi \eta(p)$	A constant force along the normal direction — replaces the time-varying barrier term

---

Equation (26) — Jacobian at the stuck point

$$J = \begin{pmatrix} 0 & I \\ -S & -k_2 I \end{pmatrix}, \quad S := k_1 I + k_3 \alpha_\Phi H(p^*)$$

Why it's here: To rigorously show the stuck equilibrium is unstable, the paper computes the Jacobian (linearisation) of Eq. (25) at the stuck point. If any eigenvalue of J has a positive real part, small perturbations grow — the equilibrium is unstable and the robot escapes.

What it is: The system matrix at the stuck equilibrium. Its eigenvalues determine stability.

Intuition: J is the $2n \times 2n$ matrix that governs how small perturbations around the stuck point evolve.

- Top-right block I: position perturbations cause velocity changes (expected).

- Bottom-left block $-S$: velocity perturbations cause acceleration changes. The matrix $S = k_1 I + k_3 \alpha_\Phi H(p^*)$ combines goal-attraction stiffness (k_1) with obstacle curvature ($H(p^*)$, the Hessian of the distance function). High obstacle curvature makes S have small or negative eigenvalues, which in turn gives J a positive eigenvalue — instability.

Variable drill-down:

Variable	Size	What it is here
J	$2n \times 2n$	**Jacobian matrix** of the frozen system (Eq. 25) linearised at $(p^*, 0)$
$\mathbf{0}$	$n \times n$	Zero block — no direct position-to-position coupling
$\mathbf{I}$	$n \times n$	Identity — position changes at rate v (trivial kinematics)
$-\mathbf{S}$	$n \times n$	The "stiffness" block — how position error maps to acceleration
$-k_2\mathbf{I}$	$n \times n$	Damping block — how velocity maps to acceleration
$\mathbf{S}$	$n \times n$	**Effective stiffness matrix**
$k_1\mathbf{I}$	$n \times n$	Goal-seeking stiffness — always positive definite (stabilising)
$k_3\alpha_\Phi\mathbf{H}(p^*)$	$n \times n$	Barrier-curvature contribution. Since $\alpha_\Phi < 0$, this term has eigenvalues $-
$\mathbf{H}(p^*)$	$n \times n$	Hessian at the stuck point. Eigenvalues = principal curvatures of obstacle surface. η is always an eigenvector with eigenvalue 0.

Eigenvalue analysis: The eigenvalues λ of J satisfy $\det(\lambda^2\mathbf{I} + k_2\lambda\mathbf{I} + \mathbf{S}) = 0$, which factors into n independent quadratics: $\lambda^2 + k_2\lambda + s_i = 0$ where s_i are eigenvalues of $\mathbf{S}$. If any $s_i < 0$, the quadratic has a positive root $\rightarrow$ instability.

Equation (27) — Instability condition (restated formally)

$$\lambda_H^i > \frac{1}{\|p^* - p_d\|}$$

Why it's here: This is the conclusion of the eigenvalue analysis of J — the formal derivation of the same condition stated informally in Eq. (15). The paper arrives here at the end of the proof of Theorem 1. Reaching this equation means the proof is complete: if obstacle curvature exceeds this threshold, the Jacobian has a positive eigenvalue, the stuck point is unstable, and the robot escapes.

What it is: The curvature threshold derived from the eigenvalue analysis of J — the final condition for the main theorem.

Intuition: Same as Eq. (15) — curved obstacles (ball, cylinder) satisfy this easily. Flat obstacles (walls) may not. This is the honest limitation the paper acknowledges: DAF doesn't guarantee escape from perfectly flat surfaces.

Derivation: Obtained by substituting $\alpha_\Phi = -\mu k_1/k_3$ and $\mu = \|p^* - p_d\|$ into $s_i = k_1 + k_3\alpha_\Phi\lambda^i_{\mathbf{H}} < 0$, giving $k_1 - k_1\|p^* - p_d\|\lambda^i_{\mathbf{H}} < 0$.

Complete Variable Reference Table

Every unique variable in the paper in one place:

Symbol	Type	Defined	Meaning
n	scalar	given	Dimension of workspace (2 or 3)
m	scalar	given	Number of obstacles
p	$\mathbb{R}^n$	state	Robot centre position
v	$\mathbb{R}^n$	state	Robot velocity
u	$\mathbb{R}^n$	design	Control acceleration (what we compute)
p_d	$\mathbb{R}^n$	given	Goal/target position
R	$\mathbb{R}_{>0}$	given	Robot radius
$\mathcal{W}$	subset of $\mathbb{R}^n$	given	Workspace (bounded region)
$\mathcal{O}_i$	subset of $\mathcal{W}$	given	i-th obstacle
$\mathcal{X}$	subset of $\mathbb{R}^n$	Eq. 2	Free space
$\mathcal{X}_\epsilon$	subset of $\mathcal{X}$	Eq. 3	Practical free space (inflated obstacles)
ϵ	$\mathbb{R}_{>0}$	Eq. 3	Safety margin
h	$\mathbb{R}_{>0}$	Assump. 1	Positive reach of free space
ρ	$\mathbb{R}_{>0}$	Assump. 2	Smoothness radius of distance function
$d(p)$	scalar	text	Effective distance: $d_{\mathcal{X}^C}(p) - (R+\epsilon)$
$\eta(p)$	$\mathbb{R}^n$, $\lvert\eta\rvert = 1$	text	Unit normal away from nearest obstacle $= \nabla d_{\mathcal{X}^C}(p)$

Symbol	Type	Defined	Meaning
$\mathbf{H}(p)$	$n \times n$ matrix	text	Hessian of distance function $= \nabla^2 d_{\mathcal{X}^C}(p)$
k_1	$\mathbb{R}_{>0}$	design	Goal-seeking gain (spring stiffness)
k_2	$\mathbb{R}_{>0}$	design	Velocity damping gain (friction)
k_3	$\mathbb{R}_{>0}$	design	Avoidance gain (obstacle braking strength)
ϵ_1	$\mathbb{R}_{>0}$	Eq. 11	Inner braking threshold
ϵ_2	$\mathbb{R}_{>0}$	Eq. 11	Outer braking threshold
$\gamma(z)$	$\mathbb{R}_{\geq 0}$	Eq. 11	Proximity scaling function
$\phi(z)$	$\mathbb{R}$	Eq. 11	Cubic blending polynomial
$D(p, v)$	scalar ≥ 0	Eq. 6	Total Rayleigh dissipation
$D_s(v)$	scalar ≥ 0	Eq. 7	Stabilisation dissipation
$D_a(p, v)$	scalar ≥ 0	Eq. 8	Avoidance dissipation
$L(p, v)$	scalar	Eq. 9	Lagrangian $= T - U$
$T(v)$	scalar ≥ 0	text	Kinetic energy $= \lvert v\rvert^2/2$
$U(p)$	scalar ≥ 0	text	Potential energy $= k_1\lvert p - p_d\rvert^2/2$
$\mathcal{E}$	set	Eq. 14	Undesired equilibrium set
μ	$\mathbb{R}_{>0}$	Eq. 14	Scalar distance from goal to stuck point
$\lambda_{\mathbf{H}}$	scalar	Eq. 15	Eigenvalue of $\mathbf{H}$ (principal curvature)
$\mathcal{L}(p, v)$	scalar ≥ 0	Eq. 19	Lyapunov function $= (k_1/2)\lvert p - p_d\rvert^2 + (1/2)\lvert v\rvert^2$
$\dot{d}$	scalar	Eq. 20	Rate of change of $d = \eta^\top v$
$\Phi(d, \dot{d})$	scalar	Eq. 22	Barrier term $= \gamma(d) \cdot \dot{d}$
$\alpha(p, v)$	scalar	Eq. 23	Bounded remainder term
α_Φ	scalar	Eq. 25	Limiting constant of Φ at stuck point $= -\mu k_1/k_3$
$\mathbf{S}$	$n \times n$ matrix	Eq. 26	Effective stiffness $= k_1\mathbf{I} + k_3\alpha_\Phi\mathbf{H}(p^*)$

Symbol	Type	Defined	Meaning
J	$2n \times 2n$ matrix	Eq. 26	System Jacobian at stuck equilibrium
s_i	scalar	Eq. 26	i-th eigenvalue of $\mathbf{S} = k_1 + k_3\alpha_\Phi\lambda_\mathbf{H}^i$

Summary: Why DAF is Better

1. **Smoother motion** — DAF brakes instead of pushing, avoiding the sudden jolts of APF.
2. **Less aggressive** — Lower peak acceleration for the same task (see Figure 6 in the paper).
3. **Fewer local minima** — APF's repulsive forces can trap the robot between obstacles; DAF's braking doesn't create artificial energy barriers.
4. **Simple to implement** — Only needs distance and direction to the nearest obstacle, plus the robot's own state.
5. **Formally safe** — Mathematical proof that the robot never collides, for any initial condition in the safe set.
6. **Works in any dimension** — The theory holds for 2D, 3D, or higher.

Summary: The Logical Chain

$$\underbrace{\text{Define world}}_{\text{Eqs. 1-4}} \longrightarrow \underbrace{\text{Model robot}}_{\text{Eq. 5}} \longrightarrow \underbrace{\text{Design energy}}_{\text{Eqs. 6-8}} \longrightarrow \underbrace{\text{Derive controller}}_{\text{Eqs. 9-13}} \longrightarrow \underbrace{\text{Find weak spots}}_{\text{Eqs. 14-15}}$$

DAF Paper — Simulation Ideas for Understanding

Ordered from simple to advanced.

---

1. Basic PD Controller (No Obstacles)

- Use only: $u = -k_1(p - p_d) - k_2 v$
- Launch from various starting positions, observe convergence
- Tune k_1 and k_2 to understand overshoot, oscillation, settling time

2. Single Circular Obstacle in 2D

- Add one circle obstacle, implement full DAF controller (Eq. 13)
- Place goal on opposite side so robot must go around
- Tests: $\gamma(d)$, normal vector η, directional braking term $\eta\eta^\top v$

3. Visualise the γ Function

- Plot $\gamma(z)$ from Eq. 11 including the cubic blending polynomial ϕ
- Verify smoothness (C^1) and blow-up as $z \to 0$
- Confirm three-zone logic with ε_1 and ε_2

4. DAF vs APF Side-by-Side (Single Obstacle)

- Implement APF controller (Eq. 18) with same gains
- Overlay trajectories from identical initial conditions
- Plot $\|u\|$, $\|v\|$, $d(t)$ over time (reproduce Figure 6)
- DAF should show lower peak acceleration, smoother paths

5. Directional Braking Demonstration

- Case 1: Robot moves parallel to a wall $\to \eta^\top v \approx 0$, avoidance term inactive
- Case 2: Robot heads straight at the wall $\to$ strong braking
- Proves understanding of why $\eta\eta^\top v$ structure matters

6. Energy and Lyapunov Function Plots

- Compute $\mathcal{L}(p, v)$ from Eq. 19 and $\dot{\mathcal{L}}$ from Eq. 20
- Verify $\mathcal{L}$ is always non-increasing
- This is the safety proof in action

7. Multiple Obstacles in 2D (Reproduce Figure 4)

- Several irregularly shaped obstacles (splines or circles/ellipses)
- Multiple initial positions, plot all trajectories
- Tests multi-obstacle distance computation and nearest-obstacle selection

8. Undesired Equilibrium (Getting Stuck)

- Place robot, circular obstacle, and goal in a perfect line: $(p - p_d) = \mu\eta(p)$
- Initialise with zero velocity at this exact point
- Add tiny perturbation $\to$ robot should escape for convex obstacles

- Direct test of Theorem 1, Item 3

9. Curvature Condition Exploration

- Obstacle A: small circle (high curvature, satisfies Eq. 15)
- Obstacle B: huge circle / gently curved wall (low curvature)
- Place goal behind each, observe escape behaviour
- Numerically compute eigenvalues of $H(p^*)$ and verify Eq. 15

10. 3D Environment (Reproduce Figure 5)

- Mix of convex and non-convex obstacles
- Tests n-dimensional distance/normal computations
- Saddle-shaped obstacles should be handled if one principal curvature satisfies Eq. 15

11. Parameter Sensitivity Study

- Sweep $k_1, k_2, k_3, \varepsilon, \varepsilon_1, \varepsilon_2$
- Record: goal reached (yes/no), min distance to obstacles, peak acceleration
- Builds intuition about feasibility conditions (Eqs. 4 and 12)

12. Algorithm 1 — Adaptive Parameter Tuning

- Create narrow corridor (two close obstacles, equidistant)
- Projection becomes non-unique $\rightarrow$ algorithm reduces $\varepsilon, \varepsilon_1, \varepsilon_2$
- Falls back to pure PD control when dissipative term can't be computed

13. LiDAR Simulation

- Replace analytical distance function with simulated range sensor
- Cast rays, find minimum range, estimate η from nearest hit
- Tests robustness to noise and discretisation

Minimum Priority Set

Priority	Simulation	What it proves
1	Single circle, DAF only (#2)	Core controller implementation
2	DAF vs APF comparison (#4)	Why DAF is better

Priority	Simulation	What it proves
3	Directional braking test (#5)	The $\eta\eta^\top v$ mechanism
4	Energy/Lyapunov plots (#6)	The safety guarantee
5	Deliberate stuck point + escape (#8)	Theorem 1 and curvature condition